I built a satellite vegetation-monitoring platform. The most valuable thing I found was a bug that produced perfectly reasonable-looking charts.

A short version of a longer write-up. Full case study and source linked at the bottom.

I spent the last stretch building the **Orbital Earth Observation Platform** — a cloud application that measures vegetation from Copernicus Sentinel-2 imagery. You pick an area and a date range; it finds the satellite scenes, masks the clouds, computes NDVI — the Normalized Difference Vegetation Index, which is just (NIR − Red) / (NIR + Red) on the near-infrared and red bands — and gives you a time series, before/after imagery, downloadable GeoTIFFs, and a provenance record for every number.

My background is Azure, containers, Terraform, CI/CD, and production operations. I wanted to find out what happens when you point that skill set at scientific data. The answer turned out to be more interesting than "you deploy a pipeline."

The bug

A deployed analysis over central Detroit showed before/after previews with different extents. The earlier image covered the northern part of the area; the later one reached the Detroit River.

It looked like a CSS problem. It wasn't.

Detroit's area of interest straddles the boundary between two Sentinel-2 tiles. A single satellite acquisition is published as **one file per tile**, so an area crossing that line matches several files representing the *same moment*. My pipeline picked one of them and required only 25% overlap with the requested area. So dates backed by the smaller tile covered **56% of the area** — and the raster read silently clipped to whatever the tile contained instead of failing.

The result: NDVI statistics computed over **751,081 pixels on some dates and 1,372,771 on others**. Different ground, plotted on the same chart as though it were the same place.

Reprocessing let me isolate the error exactly. A date that already had full coverage reproduced to four decimal places — 0.1574, unchanged. A date backed by the partial tile moved from 0.4233 to **0.3671**. The old value was inflated by 0.056 because it had been measuring only the northern, greener, less built-up half of the area.

The change that analysis originally reported was not a measurement. It was an artifact of *which pixels happened to be included*.

Why this is the interesting part

Nothing crashed. No exception, no alert, no failed test. The chart looked completely plausible — a seasonal curve with sensible values in a sensible range. If those two preview images had happened to be the same size, I would never have looked.

Three other bugs in this project had the same character:

- Removing a Sentinel-2 calibration offset left some pixels with slightly negative reflectance, which made the NDVI denominator near-zero and produced a mean of about **250,000,000**. That one at least announced itself.
- A catalog query capped at 200 results silently returned only the most recent portion of the range, so a request for **eight years of data quietly analysed four**.
- A seasonal date window used a hardcoded 365-day year and drifted by a day in leap years.

Only one of the four was loud. The rest produced output a reasonable person would have believed.

What actually caught them

Not intuition. Three unglamorous things:

Making the code assert its own invariants. Every observation in an analysis now derives from one canonical grid — same projection, same transform, same pixel footprint — and the worker *refuses to publish* an analysis whose observations disagree. The bug class is now structurally impossible rather than merely fixed.

Test fixtures adversarial enough to break it. The original tests used a single synthetic scene that fully contained the area of interest, so the truncation path was never exercised. The suite now builds two adjacent synthetic granules that each cover only part of the area, and asserts there's no seam, no gap, and identical pixel counts across dates.

Checking against a value I could predict independently. The full-coverage date reproducing *exactly* is what proved the fix was surgical rather than a wholesale change to the science.

The other half: knowing what the data means

Some things you only get from reading the mission documentation:

- Since processing baseline 04.00, Sentinel-2 reflectance carries an **additive offset**. NDVI is a ratio, so a multiplicative scale factor cancels — which makes it tempting to skip scaling entirely. An additive offset does **not** cancel. Ignore it and you get a spurious step change in any series that spans January 2022.
- The cloud-mask layer is **categorical**. Resample it with anything that averages and you invent classes that don't exist — interpolating "cloud" and "vegetation" gives you "water."
- Scene-level cloud percentage is a search filter, not a quality filter. A scene can report 5% cloud across a 110 km tile and be completely overcast above your actual 12 km area.

- Comparing years requires holding the season constant. In a temperate region NDVI swings from about 0.15 to 0.85 within a single year, while a multi-year trend is maybe 0.02–0.05. Spread eight scenes evenly across eight years and my selector picked June, February, April, May, April, October, June, May — a "trend" from that is measuring *which month each scene fell in*, by an order of magnitude.

Where it landed

The platform runs on Azure Container Apps with queue-driven workers scaling from zero, Terraform infrastructure, GitHub OIDC with no stored client secrets, and provenance documents validated against a published schema. It ships ten curated regions across five continents — Michigan, the Nile Delta, the Amazon deforestation frontier, California's Central Valley, the Okavango, the Mekong, and Doñana — each with a real analysis of live imagery.

A representative result: Southeast Michigan, April–October 2024, six observations, mean NDVI **0.444 → 0.582**. A normal deciduous growing season. Every one of those observations is a mosaic of two granules spanning two different UTM zones — which, before the fix, produced outputs in two different coordinate systems.

The platform reports that as an *observed change in greenness*. Not drought, not climate, not crop failure. It does no trend fitting and no significance testing, and the documentation says so in as many words.

What I'd take to the next role

I came in able to build the platform. What I didn't expect was how much of the work was **identifying where a software decision could silently invalidate a scientific result** — and then building the checks that make that visible.

Finding the tile bug meant noticing two images were different sizes, suspecting it wasn't cosmetic, and being able to audit raster dimensions and pixel counts to prove it was a measurement error. Fixing it meant understanding both the satellite tiling scheme and the reprojection machinery. Verifying it meant constructing a test where I already knew the answer.

The platform isn't the product. It's the thing that protects the integrity of the computation.

Live platform: oeop.net **Full technical write-up:** case-study.md **Source:** github.com/raveheart1/Orbital-Earth-Observation-Platform

Contains modified Copernicus Sentinel data, processed by ESA, accessed via the Microsoft Planetary Computer.